

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики – процессов управления

Шаго Павел Евгеньевич

Выпускная квалификационная работа

**«Математическое моделирование и реализация
планировщика процессов для гетерогенных
процессорных архитектур»**

Уровень образования: бакалавриат

Направление: 01.03.02 Прикладная математика и информатика

Основная образовательная программа СВ.5005.2022: «Прикладная математика,
фундаментальная информатика и программирование»

Научный руководитель

кандидат физ.-мат. наук, доцент

Корхов Владимир Владиславович

Рецензент

Эксперт по архитектуре и сетевому стеку

ООО НИЦ АЭРОДИСК

Анохин Юрий Владимирович

Санкт-Петербург

2026

Содержание

Введение	3
Постановка задачи	4
1 Подходы к реализации планировщиков в ОС	5
1.1 Задача планирования	5
1.2 Гетерогенные процессорные архитектуры	6
1.3 Расширяемый планировщик Linux	7
1.4 Актуальные исследования планировщиков	10
1.5 Анализ подходов к реализации планировщика	13
2 Математические модели планировщиков	15
2.1 Введение	15
2.2 Обозначения и допущения	15
2.3 Earliest Eligible Virtual Deadline First	18
2.4 Аукционная модель A1349	21
3 Реализация алгоритмов планирования	26
3.1 Реализация EEVDF на платформе sched_ext	26
3.2 Реализация аукционной модели A1349	29
4 Описание тестового окружения и экспериментального оборудования	34
4.1 Тестовое оборудование и программное окружение	34
4.2 Набор бенчмарков	35
5 Анализ результатов эксперимента	37
5.1 Результаты при лёгкой нагрузке	37
5.2 Результаты при стрессовой нагрузке	38
5.3 Результаты при умеренной нагрузке	40
5.4 Обсуждение результатов	41
Заключение	44
Список литературы	45

Введение

Постановка задачи

Объектом исследования являются алгоритмы планирования задач (процессов) в операционных системах на вычислительных устройствах с гетерогенной процессорной архитектурой (Intel Hybrid, ARM big.LITTLE и другие).

Целью работы является разработка и экспериментальная оценка планировщика, обеспечивающего более высокую эффективность вычислений на гетерогенном оборудовании по сравнению с базовым алгоритмом EEVDF ядра Linux.

Для достижения цели работы необходимо:

1. Проанализировать и определить наиболее перспективный подход к реализации планировщика.
2. Построить математическую модель, учитывающую различную вычислительную мощность ядер процессора.
3. Реализовать предложенный алгоритм с использованием инфраструктуры `sched_ext` и разработать воспроизводимый набор бенчмарков.
4. Провести эксперимент на гетерогенном оборудовании и оценить результаты в сравнении с базовым EEVDF.

1 Подходы к реализации планировщиков в ОС

1.1 Задача планирования

В современных операционных системах механизмы многозадачности являются фундаментальным инструментом управления вычислительными ресурсами. Реализация концепции псевдопараллелизма базируется на процедурах контекстного переключения, которые позволяют распределять кванты процессорного времени между потоками исполнения.

Ключевым вызовом в данном контексте является задача построения эффективного расписания, при котором порядок выполнения задач минимизирует простой оборудования и максимизирует целевые показатели эффективности. Обычно задачу построения такого расписания и дальнейшего распределения задач по ядрам процессора решает компонент непосредственно встроенный в ядро операционной системы называемый планировщиком.

С развитием операционных систем и оборудования планировщики также эволюционировали. Так, в операционной системе UNIX планировщик выбирал задачу по приоритету, а среди равных задач использовался алгоритм похожий на round-robin [1]. Позднее в ядре Linux версии 2.4 был реализован планировщик $O(n)$, который при планировании следующей задачи просматривал очередь готовых к исполнению задач и вычислял для них эвристику *goodness* [2]. В Linux 2.6 его сменил планировщик $O(1)$, в котором разделили очередь приоритетов на два массива, *active* и *expired*, что позволило выбирать следующую задачу за постоянное время [3].

Затем в версии Linux 2.6.23 произошёл переход к планировщику Completely Fair Scheduler (CFS), основанному на модели виртуального времени. Готовые к выполнению задачи хранились в красно-чёрном дереве, упорядоченном по виртуальному времени, а планировщик выбирал задачу, получившую меньше всего процессорного времени относительно своей справедливой доли [4]. В Linux 6.6 эта идея получила развитие в планировщике Earliest Eligible Virtual Deadline First (EEVDF), где вводятся отставание и виртуальный дедлайн. Допустимыми считаются задачи, у которых отставание $\text{lag} \geq 0$, среди них выбирается задача с ближайшим виртуальным дедлайном [5].

Таким образом, за последние полвека планировщик операционной системы прошёл путь от детерминированных приоритетных схем, близких по логике к round-robin, до сложных алгоритмов динамического распределения ресурсов с сильной математической базой.

1.2 Гетерогенные процессорные архитектуры

Параллельно с развитием операционных систем меняется и аппаратная платформа, на которой они исполняются. Если классические многопроцессорные системы строились на симметричном принципе (SMP), при котором все ядра обладают одинаковой производительностью и одинаковым энергопотреблением, то современные процессоры всё чаще объединяют на одном кристалле ядра разных классов. Архитектура ARM big.LITTLE впервые сделала такой подход массовым в мобильном сегменте, объединив производительные ядра (*big*) с энергоэффективными (*LITTLE*) [6]. В настольных и серверных процессорах Intel начиная с поколения Alder Lake внедрена гибридная архитектура с Performance и Efficient ядрами (P/E) [7]. Принцип не привязан к конкретной системе команд, аналогичные конфигурации прорабатываются и в экосистеме RISC-V, где появляются SoC, объединяющие на одном кристалле ядра разных классов производительности, например, гетерогенный SoC Marsellus [8].

Ключевая особенность подобных систем заключается в том, что ядра одного процессора отличаются по тактовой частоте, ширине внеочередного исполнения, размеру кэшей и удельному энергопотреблению. Одна и та же задача, выполненная на P-ядре, завершается быстрее, но обходится дороже по энергии, тогда как E-ядро обеспечивает лучшее соотношение производительность/ватт при более низкой пропускной способности. В подобных системах выбор ядра для исполнения задачи существенно влияет на итоговое поведение системы. Размещение чувствительной к задержкам задачи на E-ядре увеличивает время её обслуживания, а выполнение фоновой нагрузки на P-ядре приводит к избыточному энергопотреблению.

В ядре Linux различия между ядрами обрабатываются подсистемой Capacity Aware Scheduling (CAS), которая распознаёт неоднородные топологии и учитывает относительную мощность каждого ядра при балансировке. Поверх неё работает Energy Aware Scheduling (EAS), которая по модели энергопотребления и оценкам загрузки ядер выбирает для задачи ядро процессора с меньшим потреблением без потери пропускной способности. EAS включается лишь при выполнении ряда условий, среди которых наличие энергомодели, регулятор частоты schedutil и отсутствие SMT, поэтому применяется прежде всего на мобильных SoC семейства ARM big.LITTLE [9]. На стороне x86 для информирования планировщика о приоритете ядер задействуется механизм Intel Thread Director, транслирующий аппаратные подсказки о характере нагрузки в теги класса задачи [7]. Однако CAS и Thread Director закрывают лишь балансировку по вычислительной мощности и классификацию по профилю инструкций, тогда как отдельные очереди готовых задач для P/E-кластеров, приоритизация чувствительных к задержкам нагрузок и прикладные правила выбора

кластера остаются за пределами штатной подсистемы. Реализация подобных политик внутри fair-класса требует существенных изменений ядра, что мотивирует переход к расширяемым механизмам, рассматриваемым в следующем разделе.

1.3 Расширяемый планировщик Linux

`sched_ext` (scheduler extensions) [10] – инфраструктура (фреймворк) ядра Linux, позволяющая реализовывать политику планирования как eBPF-программу, подключаемую к интерфейсу расширяемых планировщиков во время работы системы и не требуя пересборки ядра. Фреймворк `sched_ext` был включён в основную ветку Linux в версии 6.12 и предоставляет отдельный класс планирования `ext_sched_class`, расположенный между классами `fair_sched_class` и `idle_sched_class`. При загруженном eBPF-планировщике обычные задачи переводятся из fair-класса под управление `sched_ext`.

Подход с вынесением политики планирования из ядра встречался и до `sched_ext`. В системе ghOSt [11] политика запускается как пользовательский процесс-агент, а ядро ОС исполняет полученные от агента решения о переключениях. Агент видит состояние задач через разделяемую память и отвечает за выбор задачи и процессорного ядра, тогда как ядро ОС берёт на себя лишь переключение контекста, что позволяет описывать политику на обычном языке программирования и опираться на пользовательские средства отладки.

Важно отметить, что `sched_ext` замещает только политику справедливого планирования (fair-класс). Задачи реального времени (`SCHED_FIFO`, `SCHED_RR`) и задачи с дедлайнами (`SCHED_DEADLINE`) остаются под управлением штатных классов `rt_sched_class` и `dl_sched_class` соответственно. Это ограничивает область применимости пользовательских политик, но одновременно сохраняет гарантии для критичных по времени задач.

При штатной выгрузке `sched_ext` планировщика, при наличии в нем критической ошибки или при срабатывании специального таймера ядро автоматически возвращает все задачи под управление EEVDF, что обеспечивает безопасность экспериментов с пользовательскими политиками непосредственно на работающей системе.

С точки зрения программной модели, политика планирования в `sched_ext` описывается набором обратных вызовов, объединённых в структуру `struct sched_ext_ops`. Основными обработчиками являются: `select_cpu`, вызываемый при пробуждении задачи и отвечающий за выбор ядра исполнения, `enqueue`, помещающий задачу в очередь готовых к выполнению, `dispatch`, выбирающий следующую задачу для конкретного ядра, а также `running`

и `stopping`, дающие дополнительные контрольные точки для обновления метрик (при пробуждении и снятии задачи). Дополнительные обработчики `init_task`, `exit_task`, `cpu_online` и `cpu_offline` позволяют поддерживать собственное состояние, связанное с задачами и ядрами, на протяжении всего их жизненного цикла. Отдельная группа обработчиков `cgroup_init`, `cgroup_exit`, `cgroup_move` и `cgroup_set_weight` обеспечивает интеграцию с иерархией контрольных групп: они вызываются при создании и удалении `cgroup`, перемещении задач между группами, а также при изменении веса группы, что позволяет реализовывать иерархические политики планирования.

Поведение фреймворка настраивается через флаги поля `ops->flags`. Флаг `SCX_OPS_SWITCH_PARTIAL` переводит под управление `sched_ext` только задачи с явной политикой `SCHED_EXT`, оставляя `SCHED_NORMAL`, `SCHED_BATCH` и `SCHED_IDLE` в `fair`-классе. Флаг `SCX_OPS_ENQ_LAST` определяет, попадает ли последняя выполнявшаяся задача обратно в очередь после исчерпания кванта. Флаг `SCX_OPS_KEEP_BUILTIN_IDLE` указывает ядру продолжать обновлять битовую маску простаивающих CPU при загруженной пользовательской политике. Без этого флага eBPF-программе пришлось бы вести учёт самостоятельно, тогда как с ним поиск свободного ядра доступен через штатные `kfunc`-вызовы.

Передача задач между обработчиками происходит через очереди диспетчеризации *dispatch queue* (DSQ). Помимо встроенных глобальной `SCX_DSQ_GLOBAL` и локальных по числу ядер `SCX_DSQ_LOCAL`, eBPF-программа может создавать произвольное количество собственных очередей с заданным идентификатором при помощи `scx_bpf_create_dsq`. Каждая DSQ обслуживается по принципу FIFO либо по виртуальному времени, задаваемому при вставке через `scx_bpf_dsq_insert_vtime`. В режиме виртуального времени очередь реализована как красно-чёрное дерево, упорядоченное по полю `vtime`, что обеспечивает извлечение задачи с наименьшим виртуальным временем за $O(\log n)$. Возможность создавать произвольные DSQ позволяет завести отдельную очередь на каждый кластер ядер и обслуживать её по собственному виртуальному времени. Тогда задача, помещённая в очередь нужного кластера на этапе `enqueue`, попадёт на исполнение только к ядрам этого кластера, а упорядочивание по `vtime` сохраняется внутри кластера независимо от остальных. Такое разделение существенно в контексте гетерогенных архитектур, где P/E-ядра отличаются по производительности и требуют отдельного учёта.

Для взаимодействия с ядром eBPF-программа использует набор `kfunc`-вызовов, образующих стабильный интерфейс планировщика: `scx_bpf_dsq_insert` помещает задачу в выбранную очередь, `scx_bpf_dsq_move_to_local` извлекает задачу из очереди для исполнения на текущем ядре. Состояние задач и ядер хранится в `BPF_MAP`-структурах,

а доступ к полям `task_struct` и `rq` осуществляется через механизм CO-RE (Compile Once – Run Everywhere), что обеспечивает переносимость скомпилированной программы между версиями ядра.

Для типичных нужд балансировки фреймворк предоставляет встроенный учёт простаивающих ядер: функции `scx_bpf_pick_idle_cpu` и `scx_bpf_test_and_clear_cpu_idle` позволяют без собственной реализации находить свободные CPU по битовой маске. Если обработчик `select_cpu` помещает задачу непосредственно в локальную очередь `SCX_DSQ_LOCAL` целевого ядра, последующий вызов `enqueue` для этой задачи пропускается, что образует быстрый путь пробуждения и сокращает накладные расходы на часто пробуждающихся задачах.

Собственное состояние, привязанное к конкретной задаче, удобно хранить в структуре типа `BPF_MAP_TYPE_TASK_STORAGE`, которая обеспечивает константное время доступа без хеш-поиска и автоматически освобождается при завершении задачи. Длительность кванта процессорного времени задаётся присваиванием поля `p->scx.slice` в обработчиках `enqueue` или `dispatch`, а значением по умолчанию служит `SCX_SLICE_DFL` (20 мс). Вытеснение уже исполняющейся задачи на удалённом ядре инициируется вызовом `scx_bpf_kick_cpu` с флагом `SCX_KICK_PREEMPT`.

Безопасность исполнения произвольной политики обеспечивается несколькими механизмами времени выполнения. Таймер `watchdog` отслеживает случаи, когда задача не получает процессорного времени дольше установленного порога (по умолчанию 30 секунд), и при срабатывании выгружает eBPF-планировщик. Дополнительно реализована возможность экстренного отключения пользовательской политики комбинацией клавиш `SysRq-S`.

На этапе загрузки программы среда eBPF накладывает на политику планирования ряд архитектурных ограничений, проверяемых верификатором. В программе запрещены блокирующие вызовы и динамическая аллокация памяти, любой цикл должен иметь верифицируемую верхнюю границу, а доступ к структурам ядра разрешён только через утверждённый набор `kfunc` и полей. Эти требования заметно сказываются на выборе структур данных и способов агрегирования метрик при разработке планировщика.

1.4 Актуальные исследования планировщиков

С появлением фреймворка `sched_ext` [10] исследовательская активность в области процессных планировщиков заметно возросла. Большинство современных работ используют `sched_ext` как экспериментальную платформу для быстрой проверки новых алгоритмов без модификации ядра. Диапазон применения охватывает оптимизации существующих алгоритмов, подходы машинного обучения и нестандартные постановки задач.

Tang et al. [12] предложили `sched_ext`-планировщик SRAND, в котором расчёт отставаний и виртуальных дедлайнов EEVDF заменён собственной упрощённой схемой виртуального времени и многоуровневой очередью. По завершении кванта виртуальное время задачи увеличивается на величину $(slice - p.slice) \cdot 100/p.weight$, где $slice = 20$ мс — длительность кванта, $p.slice$ — его неиспользованный остаток, $p.weight$ — вес задачи. У задач с большим весом виртуальное время растёт медленнее, что соответствует более высокому приоритету. Только что появившиеся и пробуждающиеся задачи инициализируются текущим глобальным виртуальным временем системы, которое монотонно подтягивается вверх по максимуму активных задач. Если у проснувшейся задачи виртуальное время меньше $V(t) - slice$, оно принудительно поднимается до этого порога, что не даёт долго спавшей задаче монополизировать ядро.

Полученное виртуальное время служит ключом распределения по пяти BPF-очередям с порогами 20, 50, 200 и 500 мс (`queue0–queue4`). Очереди с меньшим индексом опустошаются первыми и попадают в единую глобальную FIFO-очередь, откуда ядра напрямую извлекают следующую задачу, что заменяет обход красно-чёрного дерева EEVDF одной операцией взятия с головы. Дополнительно поддерживаются локальные очереди свободных ядер процессора с привязкой задач к ядрам, на которых они уже работали, а потоки ядра получают неограниченный квант и направляются непосредственно в локальную очередь. Политика реализована через обработчики `select_cpu`, `enqueue` и `dispatch`, время ожидания задачи в очереди ограничено порогом 5000 мс. По данным авторов, на `Lmbench` время переключения контекста сокращается на 11,83%, а на `hackbench` общая производительность растёт на 7,02% при сопоставимой с EEVDF равномерности распределения нагрузки (вариативность времени работы ядер 0,005 против 0,006 у EEVDF).

Xu et al. [13] применили `sched_ext` к задаче управляемого тестирования параллелизма (*controlled concurrency testing*, CCT) ядра Linux. Планировщик LACE сериализует конфликтующие потоки и переключает их в контролируемом порядке, автоматически вставляя точки переключения в операциях с разделяемой памятью и примитивах синхронизации, что позволяет систематически воспроизводить состояния гонки без вмешательства гипервизора. По

сравнению с фаззером SegFuzz LACE достигает на 38% большего покрытия ветвлений, снижает накладные расходы на 57% и ускоряет воспроизведение ошибок в 11,4 раза, обнаружив восемь ранее неизвестных ошибок параллелизма при объёме патчей ядра менее 25 строк. Работа показывает, что `sched_ext` пригоден не только для оптимизации производительности, но и как инструмент верификации корректности параллельного кода.

Mao et al. [14] рассмотрели задачу планирования с точки зрения обеспечения полноты данных о происхождении (*provenance*). При высоком темпе генерации событий EEVDF, LAVD и Rusty теряют от 39% до более чем 98% событий, что компрометирует гарантии эталонного монитора и открывает возможность для угрозы суперпродюсера. Планировщик Aegis строится на двухслойной архитектуре. Первый слой делит задачи на основную и несколько непервичных очередей: для непервичных задаётся время ожидания, играющее роль бюджета ресурсов, а выбор очереди происходит по наиболее *голодной* из них, что обеспечивает справедливость без явных приоритетов. Второй слой управляет переключением через DeepQ-Network с двойным вознаграждением — r_c штрафует за потерю событий и обеспечивает полноту, r_p поощряет утилизацию процессора. На бенчмарках с системами Sysdig и eAudit Aegis показал нулевую потерю событий при суммарных накладных расходах менее 0,5% в типичных режимах и около 2,44% в наиболее тяжёлых, что достигается дельта-функцией пропуска несущественных решений планирования.

Wang et al. [15] развили идею адаптивного планирования до уровня портфеля политик. Предложенный агент Adaptive Scheduling Agent (ASA) декомпозирует задачу на распознавание шаблона нагрузки и сопоставление ему планировщика из набора предобученных экспертов. Шаблон определяется ансамблевым классификатором на основе XGBoost по метрикам утилизации процессора, ввода-вывода и сетевой активности, собираемым через eBPF и `procfs`, а для подавления шумов применяется взвешенное по времени вероятностное голосование с экспоненциальным затуханием. Подготовка агента ведётся в три этапа: прототипное обучение базовой модели, динамическая калибровка стоимости переключения и обучение модели обобщения. Динамическое переключение `sched_ext`-политики выполняется на лету. ASA превосходит EEVDF в 86,4% тестовых сценариев и попадает в тройку лучших политик в 78,9% случаев, а на смешанных нагрузках обгоняет даже статический оракул за счёт переключения политик внутри одной задачи. Агент потребляет около 1,45% одного ядра и 297 МБ памяти и сохраняет качество при переносе на новые аппаратные платформы, не входившие в обучающую выборку. Работа подчёркивает ограниченность статичной политики при росте разнообразия нагрузок и оборудования.

Galín и Hedblom [16] оценили планировщик LAVD (Latency-criticality Aware Virtual Deadline) в telco-окружении Kubernetes Minikube. Поводом к работе послужила проблема задержек в облачной инфраструктуре Ericsson, резко возрастающих при загрузке процессора выше 50%, что делает оценку поведения планировщика на интервале 50–95% критически важной. LAVD изначально проектировался для игровых нагрузок и предлагается для telco в силу схожих требований к задержке. Для воспроизводимой оценки авторы реализовали многопоточный симулятор на C++, генерирующий нагрузки на основе трассировочных данных Ericsson: распределение времени обслуживания подобрано как бета-распределение, давшее наибольшее значение $p = 0,084$ по критерию Колмогорова–Смирнова, плюс отдельный генератор фоновой интерферирующей нагрузки.

В исследовании сопоставлены три планировщика: LAVD, минималистичный `scx_simple` без поддержки вытеснения и штатный EEVDF в роли базовой линии. При высокой доле фоновой интерферирующей нагрузки LAVD сокращает среднее время оборота относительно EEVDF на 6–81% и улучшает 99-й процентиль до 90%. Однако в типичных для Ericsson условиях (50% загрузки трафиком и 10% интерферирующей нагрузки) `scx_simple` стабильно обгоняет LAVD, несмотря на значительно меньшую сложность, а попытки тонкой настройки минимального и максимального кванта LAVD не дают значимых улучшений. Авторы делают вывод, что преимущество принадлежит не отдельной политике, а самой платформе `sched_ext` как средству быстрого сравнения алгоритмов и адаптации планирования к доменной специфике.

Перечисленные работы охватывают оптимизацию очередей, тестирование параллелизма, обучаемые политики и доменную адаптацию, но исходят из однородного вычислительного ресурса. Задача планирования на гетерогенных процессорах в рамках `sched_ext` остаётся значительно менее изученной. Ближе всего к ней стоит работа Van Craeynest et al. [17], в которой интенсивность обращений к памяти признана недостаточным критерием сопоставления задачи типу ядра, а оценка производительности выводится из совместного учёта ILP и MLP на уровне CPI-компонент. Предложенный механизм PIE прогнозирует поведение задачи на альтернативном ядре по CPI-стеку без её миграции и даёт прирост пропускной способности на 5,5% над современными планировщиками и на 8,7% над политиками на основе сэмплирования. Однако работа относится к 2012 году и предлагает аппаратный механизм оценки, тогда как в настоящей работе тот же учёт ведётся программно в рамках `sched_ext`. Заполнение этого пробела — учёт вычислительной мощности ядер в программной политике планирования на современных гетерогенных платформах и составляет предмет настоящей работы.

1.5 Анализ подходов к реализации планировщика

Способ реализации планировщика определяет темп исследования, цену ошибки в политике и переносимость результатов между версиями ядра. К настоящему моменту в практике закрепились четыре подхода: прямая модификация штатного fair-класса, загружаемый модуль ядра, пользовательский агент по схеме ghOSt и eBPF-программа в рамках sched_ext.

Прямая модификация штатного fair-класса обеспечивает наилучшее быстродействие, поскольку политика выполняется без промежуточных слоёв и имеет полный доступ к внутренним структурам ядра. Однако цикл разработки оказывается дорогим. Каждое изменение требует пересборки ядра (порядка 10 минут на современной машине) и перезагрузки целевой системы, ошибка в логике планирования с высокой вероятностью приводит к панике ядра, зависанию или повреждению состояния системы, а сравнение нескольких вариантов превращается в линейное накопление времени сборки и перезагрузки. Подход оправдан при подготовке итоговой реализации к включению в основную ветку Linux, но плохо подходит для итеративного исследования.

Загружаемый модуль ядра сокращает цикл итерации до секунд, поскольку вместо пересборки ядра достаточно перезагрузки модуля. Однако последствия ошибок остаются прежними, модуль исполняется с привилегиями ядра, и некорректная политика так же способна привести к панике, зависанию или повреждению состояния системы, как и встроенная реализация. Кроме того, в Linux отсутствует официальный интерфейс для замены планировщика fair-класса, поэтому модуль вынужден опираться на внутренние структуры ядра, такие как `struct sched_class` и поля `task_struct`, регулярно меняющиеся между версиями. ABI ядра в общем случае не стабилизирован, и решение оказывается жёстко привязано к конкретной версии ядра и требует сопровождения при каждом обновлении.

Пользовательский агент по схеме ghOSt [11] переносит логику планирования в обычный процесс, что обеспечивает устойчивость к ошибкам политики и широкий выбор языков реализации. Взаимодействие с ядром построено на разделяемой памяти и транзакционной передаче решений, поэтому накладные расходы заметно ниже наивной модели на системных вызовах, однако каждое решение всё равно требует синхронизации между ядром и агентом и проигрывает реализации внутри ядра на нагрузках с короткими и частыми пробуждениями. Помимо этого, ghOSt поставляется отдельным набором патчей ядра, не входящим в основную ветку, поэтому требует сопровождения при каждом обновлении ядра.

eBPF-программа в рамках sched_ext снимает основные ограничения предыдущих подходов. Программа выполняется внутри ядра, обеспечивая сопоста-

вимое со встроенной реализацией быстродействие. Безопасность гарантируется верификатором eBPF на этапе загрузки (см. раздел 1.3), а таймер watchdog при зависании политики автоматически выгружает eBPF-планировщик и возвращает систему под управление EEVDF. Подключение и выгрузка планировщика выполняются на работающей системе за миллисекунды без перезагрузки, а механизм CO-RE обеспечивает переносимость скомпилированной программы между близкими версиями ядра. Существенно и то, что с момента принятия в основную ветку sched_ext стал фактическим стандартом сравнения новых алгоритмов планирования, что упрощает воспроизведение и прямое сопоставление результатов с существующими работами.

Результаты сравнения сведены в таблицу 1.1, где знаки +, – и ± обозначают полное, отсутствующее и частичное выполнение критерия. Устойчивость к ошибкам политики здесь означает отсутствие риска паники, зависания или повреждения состояния ядра при некорректной логике планирования, а поддерживаемый интерфейс замены — наличие официального ABI для подключения внешнего планировщика без правки внутренних структур ядра.

Таблица 1.1: Сравнение подходов к реализации планировщиков

Критерий	В ядре	Модуль	ghOSt	sched_ext
Без пересборки ядра	–	+	±	+
Устойчивость к ошибкам политики	–	–	+	+
Низкие накладные расходы	+	+	–	±
Поддерживаемый интерфейс замены	–	–	±	+
Переносимость между версиями ядра	–	–	±	+

Среди рассмотренных подходов только sched_ext одновременно обеспечивает работу без пересборки ядра, устойчивость к ошибкам политики, поддерживаемый интерфейс замены и переносимость между версиями ядра, проигрывая встроенной реализации лишь по накладным расходам. Сочетание этих свойств делает возможным итеративное исследование на работающей системе и прямое сопоставление результатов с другими работами, поэтому в качестве основы для дальнейшей реализации выбран фреймворк sched_ext.

2 Математические модели планировщиков

2.1 Введение

Глава открывается сводкой обозначений и допущений, после чего описываются две модели планирования.

EEVDF [5] — алгоритм пропорционально-долевого планирования, лежащий в основе штатного планировщика Linux. Модель оперирует понятиями виртуального времени, отставания и виртуального дедлайна, обеспечивая доказуемо оптимальные границы справедливости при однородном вычислительном ресурсе. В настоящей работе она служит базовой моделью для сравнения.

Аукционная модель A1349 расширяет постановку задачи планирования на случай гетерогенного ресурса. Она опирается на динамический VCG-механизм распределения ресурсов [18, 19] и трактует выбор типа ядра как задачу аукционного распределения недолговечного блага между конкурирующими агентами. Модель учитывает различие производительностей ядер, бюджетные ограничения задач и обеспечивает совместимость стимулов.

2.2 Обозначения и допущения

В таблицах 2.1–2.4 приведена сводка обозначений, используемых в настоящей и последующих главах.

Таблица 2.1: Общие обозначения

Символ	Описание
t	Момент времени (непрерывный в EEVDF, дискретный в A1349)
q	Максимальная длительность кванта процессорного времени
w_i	Вес (приоритет) задачи i

Таблица 2.2: Обозначения модели EEVDF

Символ	Описание
$\mathcal{A}(t)$	Множество активных клиентов (задач) в момент t
$f_i(t)$	Идеальная мгновенная доля ресурса для клиента i
$S_i(t_0, t_1)$	Идеальное обслуживание клиента i на интервале $[t_0, t_1]$
$s_i(t_0, t_1)$	Фактически полученное обслуживание клиента i
t_0^i	Момент входа клиента i в систему
$\text{lag}_i(t)$	Отставание клиента i : $S_i - s_i$
$V(t)$	Виртуальное время системы
$r^{(k)}$	Длина запроса k (в единицах обслуживания)
$ve^{(k)}$	Виртуальное время доступности запроса k
$vd^{(k)}$	Виртуальный дедлайн запроса k
$u^{(k)}$	Фактически использованная часть запроса k
$r_{\max}$	Максимальная длина запроса клиента

Таблица 2.3: Обозначения модели A1349. Платформа

Символ	Описание
$\mathcal{P}, \mathcal{E}$	Множества Р-ядер и Е-ядер
K_P, K_E	Число Р-ядер и Е-ядер
K	Общее число ядер, $K = K_P + K_E$
K_P^t, K_E^t	Число свободных ядер каждого типа в момент t
η_P, η_E	Производительность Р-ядра и Е-ядра
σ	Отношение производительностей, $\sigma = \eta_P/\eta_E > 1$
c_P, c_E	Базовая стоимость одного кванта на Р-ядре и Е-ядре
γ	Отношение стоимостей, $\gamma = c_P/c_E > 1$

Таблица 2.4: Обозначения модели A1349. Задачи и механизм

Символ	Описание
$\mathcal{N}^t$	Множество активных задач в момент t
$\theta_i = (v_i, l_i)$	Скрытый тип задачи i
v_i	Ценность завершения задачи i для пользователя
$\bar{V}$	Верхняя граница ценности задачи
l_i	Требуемое число квантов задачи i на Р-ядре
m_i	Длина контракта задачи i (зависит от типа ядра)
B_i	Бюджет задачи i
κ_i	Класс обслуживания задачи i
s^t	Состояние MDP: $(\mathbf{x}^t, \mathbf{b}^t, \boldsymbol{\omega}^t)$
$\mathbf{x}^t$	Остаточные длительности контрактов на ядрах
$\mathbf{b}^t$	Остаточные бюджеты активных задач
$\boldsymbol{\omega}^t$	Внешние факторы (температура, потребление)
$a_i^t = (a_{i,P}^t, a_{i,E}^t)$	Действие: назначение задачи i на тип ядра
$\phi_P(\theta_i), \phi_E(\theta_i)$	Эффективная стоимость задачи на Р/Е-ядре
$W(s^t)$	Социальная функция благосостояния (значение Беллмана)
$W_{-i}(s^t)$	Благосостояние без участия задачи i
δ	Коэффициент дисконтирования, $\delta \in (0, 1)$
p_i^t	VCG-платёж задачи i в момент t
π	Политика планирования
$u_i(\pi)$	Полезность задачи i при политике π
$SC(\pi)$	Целевая функция с учётом штрафа за несправедливость
λ_F	Параметр баланса эффективности и справедливости
$\Psi(\pi)$	Штраф за несправедливость распределения
$\bar{W}_\kappa$	Ожидаемое будущее благосостояние для ядра типа κ

Далее перечислены допущения, общие для обеих моделей.

1. Все ядра одного типа считаются идентичными по производительности, различие существует только между Р, Е кластерами.
2. Зависимости между задачами по данным и синхронизации не учитываются моделью.
3. Планирование выполняется в онлайн-режиме, решение о назначении задачи принимается в момент её поступления без информации о будущих задачах.
4. Производительность ядер постоянна, η_P и η_E не изменяются во времени.

5. В каждый квант ядро исполняет ровно одну задачу, задача не может занимать несколько ядер одновременно.
6. Квант неделим, вытеснение происходит только на границе квантов.
7. Задача может исполняться на любом ядре (P или E), миграция между типами допускается только между квантами.
8. Контекстное переключение считается мгновенным, его стоимость не моделируется.
9. В модели A1349 задача получает полезность v_i только при полном завершении контракта длиной m_i квантов, частичное исполнение полезности не приносит.

2.3 Earliest Eligible Virtual Deadline First

Earliest Eligible Virtual Deadline First (EEVDF) [5] — модель пропорционально- долевого планирования, предложенная Ion Stoica и Hussein Abdel-Wahab.

Рассматривается один разделяемый ресурс (процессор), время на котором выделяется квантами длительности не более q . Множество клиентов (задач), активных в момент t , обозначается $\mathcal{A}(t)$, каждому клиенту i назначен положительный вес w_i . Идеальная мгновенная доля ресурса для клиента i определяется как

$$f_i(t) = \frac{w_i}{\sum_{j \in \mathcal{A}(t)} w_j}. \quad (2.1)$$

В идеализированной непрерывной модели при $q \rightarrow 0$ обслуживание $S_i(t_0, t_1)$, положенное клиенту i на интервале $[t_0, t_1]$, есть интеграл от f_i :

$$S_i(t_0, t_1) = \int_{t_0}^{t_1} f_i(\tau) d\tau. \quad (2.2)$$

Разность между положенным и фактически полученным обслуживанием задаёт центральную метрику справедливости, называемую отставанием (lag):

$$\text{lag}_i(t) = S_i(t_0^i, t) - s_i(t_0^i, t), \quad (2.3)$$

где t_0^i — момент входа клиента i в систему, s_i — фактически полученное обслуживание. Положительный lag соответствует недообслуженному клиенту, отрицательный — переобслуженному.

Виртуальное время системы $V(t)$ вводится как

$$V(t) = \int_0^t \frac{d\tau}{\sum_{j \in \mathcal{A}(\tau)} w_j}. \quad (2.4)$$

Скорость роста V обратно пропорциональна суммарному весу активных клиентов, поэтому с ростом числа клиентов виртуальное время замедляется. За одну единицу виртуального времени каждый активный клиент получает ровно w_i единиц реального обслуживания, откуда обслуживание линейно по виртуальному времени:

$$S_i(t_1, t_2) = w_i \cdot (V(t_2) - V(t_1)). \quad (2.5)$$

Каждый запрос k клиента i описывается тремя величинами: длиной $r^{(k)}$, виртуальным временем доступности $ve^{(k)}$ и виртуальным дедлайном $vd^{(k)}$. Для краткости индекс (k) опускается, когда речь идёт об одном запросе. Запрос считается *доступным* (eligible) в момент t , если $ve \leq V(t)$.

Время доступности определяет момент, начиная с которого запрос может быть обслужен. Оно выбирается так, чтобы переобслуженный клиент ожидал, пока его идеальное обслуживание сравняется с фактически полученным:

$$ve = V(t_0^i) + \frac{s_i(t_0^i, t)}{w_i}. \quad (2.6)$$

При входе клиента i в момент t_0^i первый запрос получает $ve^{(1)} = V(t_0^i)$, что соответствует нулевому отставанию в момент входа.

Виртуальный дедлайн выбирается так, чтобы за интервал $[ve, vd]$ в непрерывной модели клиент получал ровно r единиц обслуживания:

$$vd = ve + \frac{r}{w_i}. \quad (2.7)$$

При полном использовании кванта $ve^{(k+1)} = vd^{(k)}$, при частичном использовании $u^{(k)} \leq r^{(k)}$ обновление принимает вид $ve^{(k+1)} = ve^{(k)} + u^{(k)}/w_i$, что удерживает lag вблизи нуля.

Правило выбора очередной задачи состоит из двух стадий. Сначала применяется фильтр доступности $ve \leq V(t)$, затем среди доступных запросов выбирается тот, у которого виртуальный дедлайн vd минимален. Алгоритм допускает реализацию с помощью дополненного двоичного дерева поиска со сложностью $O(\log n)$ на каждую операцию (выбор, вставку, удаление).

Лемма 1 (доступность недообслуженного клиента).

Если $\text{lag}_k(t) \geq 0$ для активного клиента k , то k имеет доступный запрос в момент t .

Действительно, из $\text{lag}_k(t) \geq 0$ следует $S_k(t_0^k, t) \geq s_k(t_0^k, t)$, откуда $ve = V(t_0^k) + s_k/w_k \leq V(t_0^k) + S_k/w_k = V(t)$.

Лемма 2. В любой момент t сумма отставаний всех активных клиентов

равна нулю:

$$\sum_{i \in \mathcal{A}(t)} \text{lag}_i(t) = 0. \quad (2.8)$$

Следствие. Из лемм 1 и 2 следует, что пока множество активных клиентов непусто, существует хотя бы один доступный запрос, поэтому EEVDF является work-conserving алгоритмом, то есть ресурс не простаивает при наличии активных клиентов.

При динамических событиях (вход, выход клиентов, изменение весов) виртуальное время системы корректируется для сохранения инварианта леммы 2. При выходе клиента j в момент t

$$V(t^+) = V(t) + \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t^+)} w_i}, \quad (2.9)$$

при повторном входе клиента с отставанием $\text{lag}_j(t)$

$$V(t^+) = V(t) - \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t^+)} w_i}. \quad (2.10)$$

При изменении веса клиента j с w_j на w'_j операция эквивалентна последовательности выход–вход с новым весом:

$$V(t^+) = V(t) + \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t)} w_i - w_j} - \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t)} w_i - w_j + w'_j}. \quad (2.11)$$

В [5] рассматриваются три стратегии обработки динамических событий, различающиеся тем, как поступать с отставанием клиента при входе и выходе. Первая сохраняет отставание и корректирует V по приведённым формулам, обеспечивая наилучшую долгосрочную справедливость. Вторая обнуляет отставание при выходе. Третья допускает события только при нулевом отставании, что гарантирует непрерывность V и упрощает анализ. Границы отставания, приведённые ниже, доказаны для третьей стратегии.

Для формулировки основного результата вводится понятие стационарной системы [5].

Определение. Система называется стационарной, если все динамические события (вход, выход, изменение веса) происходят при нулевом отставании участника.

В стационарной системе виртуальное время $V(t)$ непрерывно.

Лемма 3 (граница дедлайна). В стационарной системе любой запрос активного клиента k с дедлайном d обслуживается не позднее $d + q$, где q — размер кванта.

Теорема 1 (границы отставания) [5]. В стационарной системе для любого активного клиента k отставание ограничено как

$$-r_{\max} < \text{lag}_k(t) < \max(r_{\max}, q), \quad (2.12)$$

где $r_{\max}$ — максимальная длина запроса клиента k . Обе границы асимптотически точны.

Следствие 2 (равномерный квант) [5]. Если все запросы клиента k не превышают кванта, $r^{(k)} \leq q$, то

$$-q < \text{lag}_k(t) < q. \quad (2.13)$$

Лемма 5 (оптимальность) [5]. Для любого пропорционально-долевого алгоритма с квантами размера q отставание любого клиента ограничено снизу $-q$ и сверху q .

Доказательство строится на примере n клиентов с равными весами: клиент, получивший первый квант, имеет $\text{lag} = q/n - q \rightarrow -q$ при $n \rightarrow \infty$, а получивший последний, $\text{lag} = q - q/n \rightarrow q$. Ни один алгоритм не может дать границу лучше $(-q, q)$, и EEVDF достигает этой границы.

В стационарной системе с постоянным составом клиентов EEVDF эквивалентен алгоритму Earliest Deadline First (EDF). Фильтр доступности ($ve \leq V(t)$) отличает EEVDF от алгоритмов справедливой очередизации (Packet-by-Packet Fair Queueing), ограничивая отставание до $O(1)$ вместо $O(n)$ [5].

2.4 Аукционная модель A1349

Базовая модель EEVDF предполагает однородный вычислительный ресурс с фиксированной мощностью каждого ядра. В современных гетерогенных процессорах (Intel P/E, ARM big.LITTLE) разница в производительности между классами ядер достигает двух-трёх раз. Прямая трактовка таких ядер как одинакового ресурса приводит к неоптимальному распределению задач: чувствительные к латентности задачи могут оказаться на медленных ядрах, а фоновые, напротив, на быстрых. Поэтому для гетерогенной платформы в настоящей работе предлагается аукционная модель планирования A1349, в которой выбор ядра трактуется как задача распределения недолговечного ресурса между конкурирующими агентами. Формализация опирается на модель динамического механизма Ryuji Sano [18] для ресурсов с различной длительностью использования и на результаты Vincent Leon и S. Rasoul Etesami [19] по онлайн-обучению в динамических VCG-механизмах.

Платформа и агенты.

Гетерогенная платформа описывается двумя множествами ядер: $\mathcal{P} = \{P_1, \dots, P_{K_P}\}$, ядра высокой производительности (P-cores), и $\mathcal{E} = \{E_1, \dots, E_{K_E}\}$, энергоэффективные ядра (E-cores). Время разделено на дискретные кванты $t = 1, 2, \dots$. В каждый момент времени каждое ядро представляет собой *недолговечный* (perishable) ресурс: неиспользованный квант теряется безвозвратно.

Каждая задача $i \in \mathcal{N}^t$ выступает в роли агента-покупателя со скрытым типом $\theta_i = (v_i, l_i)$, где $v_i \in [0, \bar{V}]$, ценность завершения задачи для пользователя, $l_i \in \{1, \dots, L\}$, требуемое число квантов на Р-ядре. Поскольку Р- и Е-ядра разделяют общий набор инструкций, задача может быть назначена на любой класс ядер; выбор типа является решением механизма, а не задачи. На Е-ядре длина задачи увеличивается в $\sigma > 1$ раз и составляет $\lceil l_i \cdot \sigma \rceil$ квантов, где $\sigma = \eta_P / \eta_E$ выражает отношение производительностей. Задача i получает полезность v_i только при исполнении полного контракта длиной m_i квантов.

Каждой задаче назначен бюджет $B_i > 0$, определяемый её классом обслуживания (real-time, interactive, batch, background). Задача может выполняться, пока суммарные платежи $\sum_s p_i^s$ не превышают B_i . Принадлежность к классу κ_i является экзогенным параметром и не входит в скрываемый тип θ_i .

MDP-формулировка.

Процесс планирования формализуется как марковский процесс принятия решений (MDP). Состояние в момент t

$$s^t = (\mathbf{x}^t, \mathbf{b}^t, \boldsymbol{\omega}^t), \quad (2.14)$$

где $\mathbf{x}^t \in \{0, \dots, L-1\}^K$, остаточные длительности контрактов на каждом из $K = K_P + K_E$ ядер, $\mathbf{b}^t$, остаточные бюджеты активных задач, $\boldsymbol{\omega}^t$, внешние факторы (температура, потребление). Действие a^t задаёт для каждой задачи распределение по типам ядер $a_i^t = (a_{i,P}^t, a_{i,E}^t) \in \{0, 1\}^2$, $a_{i,P}^t + a_{i,E}^t \leq 1$, с ограничениями

$$\sum_{i \in \mathcal{N}^t} a_{i,P}^t \leq K_P^t, \quad \sum_{i \in \mathcal{N}^t} a_{i,E}^t \leq K_E^t, \quad (2.15)$$

где K_P^t, K_E^t , число свободных ядер каждого типа.

Следующее состояние определяется функцией перехода $s^{t+1} = G(s^t, a^t, \mathcal{N}^{t+1})$, где $\mathcal{N}^{t+1}$, множество вновь прибывших задач. Ядро k , назначенное задаче i с длиной контракта $m_\kappa(l_i)$, переходит в состояние $x_k^{t+1} = m_\kappa(l_i) - 1$; для остальных ядер $x_k^{t+1} = \max(x_k^t - 1, 0)$. Остаточные бюджеты уменьшаются на величину платежа: $b_i^{t+1} = b_i^t - p_i^t$.

Контракты и стоимость.

Базовая стоимость одного кванта на ядре типа $\kappa \in \{P, E\}$ обозначается c_P, c_E , причём $c_P > c_E$ (отношение $\gamma = c_P / c_E$ введено в таблице обозначений).

Полная стоимость контракта для задачи i :

$$\text{cost}(i, P) = c_P \cdot l_i, \quad \text{cost}(i, E) = c_E \cdot [l_i \cdot \sigma]. \quad (2.16)$$

Соотношение нелинейно: при $[l_i \sigma]/l_i < \gamma$ Е-ядро оказывается дешевле, иначе дороже. Это делает выбор типа ядра нетривиальным.

Эффективная стоимость задачи на каждом типе ядра с учётом стоимости квантов:

$$\phi_P(\theta_i) = v_i - c_P \cdot l_i, \quad \phi_E(\theta_i) = v_i - c_E \cdot [l_i \cdot \sigma]. \quad (2.17)$$

Расширенная социальная функция благосостояния удовлетворяет уравнению Беллмана:

$$W(s^t) = \max_{a^t} \left[\sum_{i \in \mathcal{N}^t} (a_{i,P}^t \phi_P(\theta_i) + a_{i,E}^t \phi_E(\theta_i)) + \delta \cdot \mathbb{E}[W(s^{t+1})] \right], \quad (2.18)$$

где $\delta \in (0, 1)$, коэффициент дисконтирования. В настоящей работе используется дисконтированная формулировка, следуя Ryuji Sano [18]; результаты Vincent Leon и S. Rasoul Etesami [19] по онлайн-обучению распространяются на формулировку со средним вознаграждением (average-reward MDP), что обеспечивает применимость полученных гарантий и при бесконечном горизонте без дисконтирования.

Целевая функция оптимизации включает штраф за несправедливость:

$$\text{SC}(\pi) = W(\pi) - \lambda_F \cdot \Psi(\pi), \quad \Psi(\pi) = \text{Var}_i \left[\frac{u_i(\pi)}{v_i} \right], \quad (2.19)$$

где u_i , полезность задачи i при политике π , $\lambda_F \geq 0$, параметр баланса между общей эффективностью и равномерностью распределения.

Динамический VCG-механизм.

Обозначим $m_\kappa(l)$ длину контракта задачи на ядре типа κ : $m_P(l) = l$, $m_E(l) = [l \cdot \sigma]$.

Платёж задачи i , получившей ядро типа κ в момент t , определяется по правилу VCG (Викри–Кларка–Гровса) как разность между социальным благом без её участия и вкладом в текущее распределение:

$$p_i^t = W_{-i}(s^t) - W(s^t) + \phi_\kappa(\theta_i). \quad (2.20)$$

Здесь $W_{-i}(s^t)$ обозначает оптимальное значение функции Беллмана той же MDP, в которой оптимизация проводится по всем задачам, кроме i . Интуитивно задача оплачивает экстерналию, налагаемую ею на остальные активные

задачи. В частном случае одного свободного ядра платёж сводится к

$$p_{i^*} = \phi_\kappa(\theta_j) + (\delta^{m_\kappa(l_j)} - \delta^{m_\kappa(l_{i^*})})\bar{W}_\kappa, \quad (2.21)$$

где j , следующая по эффективной стоимости задача, $\bar{W}_\kappa$, ожидаемое будущее благосостояние в зависимости от типа ядра.

Онлайн-обучение механизма.

При неизвестных распределениях типа задач и переходного ядра MDP механизм обучается онлайн. Горизонт времени T разбивается на эпизоды $k = 1, 2, \dots$ возрастающей длительности. Каждый эпизод k состоит из двух фаз: фазы перемешивания (mixing) длительностью $d^{[k]} = \frac{\log k}{\alpha S}$ квантов, в течение которой текущая политика $\pi^{[k]}$ применяется без сбора платежей для приближения марковской цепи к стационарному распределению, и стационарной фазы длительностью $l^{[k]} = \frac{\max\{4, \sqrt{k}\} \log(|\mathcal{A}||\mathcal{S}|k/\zeta)}{\alpha \delta}$ квантов, в которой та же политика $\pi^{[k]}$ применяется с одновременным сбором платежей и оценок вознаграждений. По завершении эпизода оценки $\hat{R}, \hat{P}$ строятся по принципу верхней доверительной границы (UCB/LCB) и обновляют политику решением $(n+1)$ линейных программ. Vincent Leon и S. Rasoul Etesami [19] доказывают сублинейные оценки сожаления (regret) по социальному благосостоянию и платежам порядка $\tilde{O}(T^{2/3})$; при известном горизонте T выбор $\epsilon \leq O(T^{-1/3})$ даёт верхнюю границу $\tilde{O}(T^{2/3})$, совпадающую с нижней границей $\Omega(T^{2/3})$, установленной для эпизодических MDP, с точностью до логарифмического множителя.

Свойства механизма.

Теорема 2 (совместимость стимулов, Ryuji Sano [18], Theorem 1).

Динамический прямой механизм является совместимым по стимулам тогда и только тогда, когда для каждой задачи i : (1) вероятность получения ядра $\alpha_i(v_i, l_i)$ не убывает по v_i при фиксированном l_i ; (2) $\int_0^{v_i} \alpha_i(\nu, l_i) d\nu$ не возрастает по l_i при фиксированном v_i ; (3) существует $\underline{\Pi}_i$, не зависящее от l_i , такое что $\Pi_i(v_i, l_i) = \underline{\Pi}_i + \int_0^{v_i} \alpha_i(\nu, l_i) d\nu$.

При фиксированном классе обслуживания κ_i правдивое сообщение типа (v_i, l_i) является доминирующей стратегией для агента. Межклассовая совместимость обеспечивается тем, что класс задаётся ядром ОС вне канала сообщений механизма.

Следствие 1 (индивидуальная рациональность, Ryuji Sano [18]; Vincent Leon и S. Rasoul Etesami [19], Theorem 1).

Полезность задачи при участии в механизме неотрицательна, $u_i(\pi^, p^*) = W(\pi^*; R) - W_{-i}(\pi^*; R_{-i}) \geq 0$. Каждая задача получает неотрицательную полезность, и участие в аукционе является для неё выгодным.*

Предложение 1 (бюджетная допустимость).

Если VCG -платёж превышает остаточный бюджет, $p_i^*(s, a) > B_i^t$, задача не получает ядро в данном кванте. Модифицированное распределение $\tilde{a}^t = \arg \max_a \sum_{i: p_i \leq B_i^t} [a_{i,P} \phi_P(\theta_i) + a_{i,E} \phi_E(\theta_i)] + \delta \mathbb{E}[W(s^{t+1})]$ сохраняет совместимость стимулов внутри класса, поскольку ограничение реализуется изменением правила распределения, а не платежа.

Таким образом, модель A1349 расширяет идею пропорционально-долевого планирования аукционным механизмом, учитывающим гетерогенность ядер, бюджетные ограничения задач и динамику системы во времени.

3 Реализация алгоритмов планирования

3.1 Реализация EEVDF на платформе sched_ext

Данный раздел описывает реализацию математической модели EEVDF (раздел 2.3) в виде eBPF-программы для платформы sched_ext. Реализация служит базовой линией для экспериментального сравнения с аукционной моделью A1349 и демонстрирует прямое соответствие между формулами модели и обработчиками фреймворка.

Переменные состояния. Глобальное состояние системы хранится в BPF-карте типа `BPF_MAP_TYPE_ARRAY` с единственным элементом структуры `eevdf_ctx`, содержащей два поля: `vtime_now` — системное виртуальное время $V(t)$ (уравнение 4), масштабированное коэффициентом `SCALE = 1000` для обеспечения целочисленной точности без потери младших разрядов, и `total_weight` — суммарный вес активных задач $\sum_{j \in A(t)} w_j$, используемый в знаменателе при продвижении V .

Состояние отдельной задачи хранится в карте типа `BPF_MAP_TYPE_TASK_STORAGE`, что обеспечивает обращение за $O(1)$ без хеш-поиска и автоматическое освобождение при завершении задачи. Структура `task_ctx` содержит виртуальное время доступности `ve` (соответствует $ve^{(k)}$ в уравнении 6), виртуальный дедлайн `vd` ($vd^{(k)}$, уравнение 7) и флаг нахождения в очереди `on_rq`. Все временные величины хранятся в масштабированных единицах.

Единственная очередь диспетчеризации `SHARED_DSQ` создаётся при инициализации и упорядочена по виртуальному дедлайну vd : при вставке через `scx_bpf_dsq_insert_vtime` ядро использует поле `vtime` как ключ красного дерева, что реализует правило выбора EEVDF — среди доступных запросов извлекается тот, чей vd минимален.

Отображение обработчиков на модель. В таблице 3.1 приведено соответствие между обратными вызовами sched_ext и шагами алгоритма EEVDF.

Ключевые аспекты реализации.

Атомарное продвижение $V(t)$. Системное виртуальное время обновляется при каждом снятии задачи с процессора (обработчик `stopping`). Поскольку на многоядерной системе несколько ядер могут одновременно вызвать `stopping`, продвижение V выполняется через атомарную операцию

Таблица 3.1: Соответствие обработчиков sched_ext модели EEVDF

Обработчик	Реализуемый шаг модели
select_cpu	Быстрый путь пробуждения: если $ve \leq V(t)$ (лемма 1) и найдено свободное ядро, задача помещается в SCX_DSQ_LOCAL без прохода через общую очередь
enqueue	Двусторонний ограничитель отставания (теорема 1), вычисление $vd = ve + q/w_i$ (уравнение 7), вставка в SHARED_DSQ по vd
dispatch	Извлечение головы DSQ — задачи с наименьшим vd среди доступных
stopping	Продвижение ve по формуле $ve += u/w_i$ (обновление при частичном использовании), продвижение $V(t)$ на $\Delta V = u / \sum w_j$ (уравнение 4)
enable	Вход задачи: $ve := V(t)$, $\sum w_j += w_i$ (уравнение 10 при нулевом lag)
disable	Выход задачи: коррекция $V(t^+) = V(t) + \text{lag} / \sum w_{j \neq i}$ (уравнение 9)
set_weight	Изменение веса: $V(t^+)$ по уравнению 11 (выход-вход с новым весом)

__sync_fetch_and_add. Величина приращения

$$\Delta V = \frac{u \cdot \text{SCALE}}{\sum_j w_j}, \quad (3.1)$$

где u — фактически использованная часть кванта (SCX_SLICE_DFL — p->scx.slice), соответствует дискретному аналогу скорости $dV/dt = 1 / \sum w_j$ из уравнения 4. Масштабирование на SCALE обеспечивает сохранение дробных долей при целочисленном делении.

Двусторонний ограничитель отставания. При постановке задачи в очередь (обработчик enqueue) применяется ограничение

$$\max(V(t) - q_v, ve) \leq ve' \leq V(t) + q_v,$$

где $q_v = q \cdot \text{SCALE} / w_i$ — виртуальная стоимость одного кванта. Нижняя граница соответствует теореме 1: задача, долго не получавшая обслуживания, ограничивается одним квантом «накопленного преимущества», что предотвращает монополизацию ресурса. Верхняя граница аналогично не позволяет задаче оказаться слишком далеко в будущем виртуального времени. В терминах модели оба ограничения следуют из границ $-r_{\max} < \text{lag}_k(t) < \max(r_{\max}, q)$ теоремы 1.

Обработка динамических событий. При входе задачи (enable) её виртуальное время доступности инициализируется как $ve = V(t)$, что соответствует

нулевому отставанию в момент входа. Суммарный вес обновляется атомарно. При выходе (**disable**) виртуальное время корректируется по уравнению 9: $V(t^+) = V(t) + \text{lag} / \sum w_{j \neq i}$, где $\text{lag} = V(t) - ve$ (положительный lag означает недообслуженность). При изменении веса (**set_weight**) применяется двухшаговая коррекция по уравнению 11, эквивалентная последовательности выход–вход с новым весом.

Таким образом, реализация сохраняет все инварианты модели EEVDF при ограничениях среды eBPF: отсутствии блокирующих вызовов, конечных циклах и фиксированных структурах данных. Единственное отклонение от теоретической модели состоит в замене непрерывного продвижения $V(t)$ дискретными приращениями на границах квантов, что не нарушает границ теоремы 1, поскольку q конечно.

3.2 Реализация аукционной модели A1349

Данный раздел описывает реализацию аукционной модели A1349 (раздел 2.4) в виде eBPF-планировщика `scx_auction` для платформы `sched_ext`. Реализация расширяет базовый механизм виртуального времени EEVDF аукционным распределением задач между кластерами P/Е-ядер, бюджетным учётом и аппроксимацией VCG-платежа.

Архитектура очередей диспетчеризации. При инициализации создаются три основных очереди и набор вспомогательных:

- `AUCTION_DSQ_P` — очередь задач, назначенных на P-кластер (множество $\mathcal{P}$ модели);
- `AUCTION_DSQ_E` — очередь задач, назначенных на E-кластер (множество $\mathcal{E}$);
- `AUCTION_DSQ_STARVED` — очередь задач с исчерпанным бюджетом (нарушение бюджетного ограничения B_i);
- `AUCTION_DSQ_PERCPU_BASE + cpu` — набор привязанных к ядрам очередей для длительных задач, обеспечивающий кэш-локальность.

Все DSQ упорядочены по виртуальному дедлайну и обслуживаются по правилу наименьшего *vd*. Двойная структура `DSQ_P/DSQ_E` реализует отдельные очереди для каждого кластера, описанные в разделе 1.3, что позволяет учитывать различие производительности ядер при диспетчеризации.

Переменные состояния. В таблице 3.2 приведены основные переменные состояния и их соответствие символам модели.

Таблица 3.2: Переменные состояния планировщика A1349

Переменная	Символ модели	Описание
<code>vtime_now</code>	$V(t)$	Системное виртуальное время
<code>total_weight</code>	$\sum w_j$	Суммарный вес активных задач
<code>dsq_phi_hi_p</code>	$\varphi_{hi,P}$	Оценка максимального ϕ_P в P-кластере
<code>dsq_phi_hi_e</code>	$\varphi_{hi,E}$	Оценка максимального ϕ_E в E-кластере
<code>budget</code>	B_i^t	Остаточный бюджет задачи
<code>budget_max</code>	B_i	Максимальный бюджет ($w_i \cdot \text{BUDGET_MUL}$)
<code>len_est_ns</code>	l_i	EWMA длительности исполнения (прокси для l_i)
<code>on_p_type</code>	κ_i	Текущий кластер назначения

Глобальная конфигурация (структура `auction_ctx`) заполняется из пространства пользователя при загрузке планировщика и содержит параметры

платформы: `max_capacity` (η_P), `min_capacity` (η_E), `cost_p` (c_P), `cost_e` (c_E), число P- и E-ядер (K_P , K_E). Отношение производительностей $\sigma = \eta_P/\eta_E$ вычисляется из `max_capacity` / `min_capacity`, а стоимости автоматически калибруются в пространстве пользователя так, чтобы $\gamma = c_P/c_E = \sigma$.

Отображение обработчиков на модель. В таблице 3.3 приведено соответствие обработчиков шагам аукционного механизма.

Таблица 3.3: Соответствие обработчиков `sched_ext` аукционной модели

Обработчик	Реализуемый шаг модели
<code>select_cpu</code>	Быстрый путь: поиск свободного ядра, сохранение <code>prev_cpu</code> для кэш-привязки, прямая вставка в <code>SCX_DSQ_LOCAL</code> при наличии простаивающего ядра
<code>enqueue</code>	Вычисление ϕ_P , ϕ_E (уравнение 16); аукционное решение $\kappa^* = \arg \max_{\kappa} \phi_{\kappa}$; бюджетная проверка B_i^t ; обновление $\varphi_{hi,\kappa}$; вставка в DSQ по vd
<code>dispatch</code>	Извлечение задачи из кластерной DSQ для текущего ядра, кросс-кластерное заимствование, обслуживание STARVED
<code>running</code>	Продвижение глобального $V(t)$ до ve задачи (монотонность)
<code>stopping</code>	Продвижение ve и $V(t)$ с учётом мощности ядра; вычисление VCG-платежа p_i^t (уравнение 19); списание бюджета; обновление EWMA длительности
<code>enable</code>	Инициализация бюджета $B_i = w_i \cdot \text{BUDGET_MUL}$; вход в систему виртуального времени
<code>disable</code>	Коррекция $V(t)$ при выходе задачи (уравнение 9); освобождение состояния

Вычисление эффективной стоимости. Модель определяет эффективную стоимость задачи на ядре типа κ как (уравнение 16):

$$\phi_P(\theta_i) = v_i - c_P \cdot l_i, \quad \phi_E(\theta_i) = v_i - c_E \cdot \lceil l_i \cdot \sigma \rceil.$$

В реализации используется непрерывная формулировка в наносекундах, устраняющая потерю точности при квантизации коротких задач:

$$\phi_P = w_i \cdot s - c_P \cdot \hat{l}_{ns}, \quad \phi_E = w_i \cdot s \cdot \frac{1}{\sigma} - c_E \cdot \hat{l}_{ns},$$

где $w_i = \text{p->scx.weight}$ (прокси для v_i), $s = \text{AUCTION_SLICE_P}$ (длительность кванта), $\hat{l}_{ns} = \text{len_est_ns}$ (экспоненциально взвешенное скользящее среднее фактической длительности исполнения, $\alpha = 1/8$). Множитель $1/\sigma$ на стороне значения ϕ_E отражает тот факт, что E-ядро доставляет в σ раз меньше полез-

ной работы за тот же квант, а потому ценность кванта на Е-ядре дисконтирована.

Условие предпочтения Р-кластера $\phi_P \geq \phi_E$ сводится к неравенству

$$w_i \cdot s \cdot \frac{\sigma - 1}{\sigma} \geq (c_P - c_E) \cdot \hat{l}_{ns},$$

из которого следует, что короткие задачи с большим весом направляются на Р-ядра, а длительные вычислительные задачи — на Е-ядра.

Бюджетный механизм. Каждой задаче назначается бюджет в виде корзины токенов (token bucket), моделирующей ограничение B_i из раздела 2.4. Максимальный бюджет пропорционален весу: $B_i^{\max} = w_i \cdot \text{BUDGET_MUL}$. Пополнение происходит при пробуждении задачи пропорционально времени сна:

$$\Delta B_i = \min(\Delta t_{\text{idle}}, T_{\max}) \cdot \frac{w_i}{\text{REPLENISH_DIV}},$$

где $T_{\max} = 1$ с — ограничитель, предотвращающий переполнение после длительного сна. Задача считается истощённой (starved), если $B_i < B_i^{\max} / \text{STARVE_FRAC}$, и перенаправляется в очередь `AUCTION_DSQ_STARVED` с дополнительным штрафом к виртуальному дедлайну, пропорциональным дефициту ϕ :

$$vd += \frac{\max(0, -\phi_\kappa)}{\text{STARVE_PHI_DIV}}.$$

Это реализует требование бюджетной допустимости из раздела 2.4: задача, чьи суммарные платежи превышают бюджет, не получает ядро в текущем кванте и обслуживается с пониженным приоритетом.

VCG-платёж. Точное вычисление $W_{-i}(s^t)$ из уравнения 19 требует решения MDP без участия задачи i , что невозможно в eBPF-обработчике за ограниченное время. Реализация использует аппроксимацию в форме резервной цены Викри:

$$p_i^t = \max\left(c_\kappa \cdot \frac{t_{\text{consumed}}}{s}, \varphi_{\text{hi},\kappa} \cdot \frac{t_{\text{consumed}}}{s^2}\right), \quad (3.2)$$

где t_{consumed} — фактически использованная часть кванта, s — длительность предоставленного кванта, $\varphi_{\text{hi},\kappa}$ — скользящая оценка максимального положительного ϕ в кластере κ . Первый член реализует резервную цену Майерсона (posted cost), второй — экстерналию, налагаемую задачей на лучшую альтернативу в очереди.

Оценка $\varphi_{\text{hi},\kappa}$ обновляется при каждой постановке в очередь по правилу:

$$\varphi_{\text{hi},\kappa} \leftarrow \begin{cases} \phi_\kappa(\theta_i), & \text{если } \phi_\kappa(\theta_i) > \varphi_{\text{hi},\kappa}, \\ \frac{15 \cdot \varphi_{\text{hi},\kappa} + \max(0, \phi_\kappa)}{16}, & \text{иначе (EWMA, } \alpha = 1/16). \end{cases}$$

Мгновенный подъём при новом максимуме и медленное затухание обеспечивают сходимость к верхней порядковой статистике ϕ в стационарном режиме, что аппроксимирует W_{-i} в однослотовом случае (уравнение 20).

Маршрутизация с гистерезисом. Решение аукциона $\kappa^* = \arg \max_{\kappa} \phi_{\kappa}$ пересматривается только при пробуждении задачи (SCX_ENQ_WAKEUP). Вытесненные задачи сохраняют текущий кластер, что предотвращает миграцию $P \leftrightarrow E$ в середине исполнения. Дополнительно реализована зона нечувствительности (гистерезис): переключение кластера происходит, только если разность $|\phi_P - \phi_E|$ превышает 12,5% от $\max(\phi_P, \phi_E)$. Это предотвращает осцилляцию задач с близкими значениями ϕ вблизи границы кроссовера и соответствует области бездействия (inaction region) в пороговых контрактных аукционах. Поверх гистерезиса реализован временной охлаждающий период: миграция между кластерами подавляется в течение 2 мс после предыдущего переключения.

Асимметричное перенаправление. Чистый $\arg \max \phi_{\kappa}$ не учитывает числа ядер в кластерах: на платформе с $K_P = 8$, $K_E = 16$ все задачи с $\phi_P > \phi_E$ направлялись бы на Р-кластер, оставляя Е-ядра простаивающими. Реализация вводит перенаправление (spill) на основе нормализованной глубины очереди: задача, назначенная на Р-кластер, переводится в Е-кластер, если

$$Q_P \geq K_P \quad \text{и} \quad \frac{Q_P}{K_P} > \frac{Q_E}{K_E},$$

где Q_{κ} — текущая глубина очереди AUCTION_DSQ_ κ . Условие сохраняет работоспособность (work conservation): перенаправление происходит только при наличии свободных Е-ядер и относительном перенаселении Р-кластера.

Диспетчеризация и заимствование. Обработчик dispatch реализует многоуровневый приоритет извлечения:

1. привязанная к ядру очередь (PERCPU_DSQ) — длительные задачи, сохраняющие кэш-локальность;
2. кластерная очередь для текущего типа ядра (DSQ_P или DSQ_E);
3. кросс-кластерное заимствование: если локальная очередь пуста, ядро извлекает задачу из противоположного кластера;
4. очередь STARVED — последний приоритет, задачи с исчерпанным бюджетом.

Кросс-кластерное заимствование гарантирует, что ни одно ядро не простаивает при наличии готовых задач, — свойство work conservation, критичное для модели A1349, где неиспользованный квант является потерянным ресурсом.

Виртуальное время с учётом мощности ядра. В отличие от базовой реализации EEVDF, где все ядра продвигают виртуальное время одинаково,

планировщик A1349 нормализует приращение по мощности ядра:

$$\Delta ve_i = \frac{u \cdot \text{cap}(\text{cpu}) \cdot \text{SCALE}}{\text{CAPACITY_SCALE} \cdot w_i}, \quad (3.3)$$

где $\text{cap}(\text{cpu})$ — аппаратная мощность текущего ядра из `/sys/.../cpu_capacity`. Быстрое Р-ядро продвигает ve быстрее, чем медленное Е-ядро за тот же реальный квант, что сохраняет справедливость в виртуальном времени: задача, исполнявшаяся на Р-ядре, «потребляет» больше виртуального ресурса за единицу реального времени, что соответствует различию производительностей $\eta_P > \eta_E$.

Пространство пользователя. Программа `scx_auction.c` выполняет функции агента конфигурации: считывает мощности ядер из `sysfs`, вычисляет σ и автоматически калибрует $c_E = c_P \cdot \eta_E / \eta_P$ так, чтобы отношение стоимостей $\gamma = c_P / c_E$ совпадало с σ . Агент периодически обновляет карты BPF при изменении топологии (hot-plug). Разделение на конфигурационную карту (`global_data`, доступную пространству пользователя) и карту исполнительного состояния (`runtime_data`, доступную только BPF) исключает гонки между периодическим обновлением параметров и продвижением виртуального времени в обработчиках.

Таким образом, реализация аукционной модели A1349 сохраняет ядро EEVDF (виртуальное время, дедлайны, ограничители отставания) и расширяет его механизмами аукционного распределения между кластерами, бюджетного учёта и аппроксимации VCG-платежа, работающими в пределах ограничений верификатора eBPF.

4 Описание тестового окружения и экспериментального оборудования

4.1 Тестовое оборудование и программное окружение

Экспериментальная оценка планировщика проведена на настольной платформе с гетерогенным процессором Intel Core Ultra 7 265K (кодовое название Arrow Lake-S). Процессор объединяет на одном кристалле два класса ядер: 8 Р-ядер микроархитектуры Lion Cove с максимальной тактовой частотой 5,5 ГГц и 16 Е-ядер микроархитектуры Skymont с максимальной тактовой частотой 4,6 ГГц. Технология Hyper-Threading отсутствует, каждое ядро исполняет один аппаратный поток, что даёт 24 потока при 24 физических ядрах. Характеристики платформы сведены в таблицу 4.1.

Таблица 4.1: Характеристики тестовой платформы

Параметр	Значение
Процессор	Intel Core Ultra 7 265K (Arrow Lake-S)
Р-ядра	8 × Lion Cove, до 5,5 ГГц
Е-ядра	16 × Skymont, до 4,6 ГГц
Потоков на ядро	1 (без Hyper-Threading)
Всего потоков	24
Кэш L2	36 МБ
Кэш L3 (разделяемый)	30 МБ
Оперативная память	32 ГБ DDR5
Операционная система	Linux 7.0
Планировщик по умолчанию	EEVDF (fair-класс)
Фреймворк	sched_ext

Выбор платформы обусловлен прямым соответствием между аппаратной топологией и математической моделью. Процессор содержит два кластера ядер с различной производительностью, что в точности воспроизводит допущение модели A1349 о множествах $\mathcal{P}$ и $\mathcal{E}$ с отношением производительностей $\sigma = \eta_P/\eta_E > 1$. Отсутствие Hyper-Threading исключает влияние разделяемых ресурсов между аппаратными потоками внутри одного ядра и упрощает интерпретацию результатов, поскольку каждая задача получает полный набор исполнительных блоков ядра без конкуренции на уровне SMT.

В качестве операционной системы используется ядро Linux версии 7.0 с включённой поддержкой sched_ext. Базовым планировщиком fair-класса служит EEVDF, который в экспериментах выступает контрольной линией. Предложенный планировщик s4 загружается поверх sched_ext без перезагрузки и перес-

борки ядра. Регулятор частоты установлен в режим **performance**, что фиксирует тактовую частоту на максимальном значении и устраняет влияние динамического масштабирования на измерения.

4.2 Набор бенчмарков

Для оценки планировщика используется набор из трёх бенчмарков, каждый из которых генерирует характерный профиль нагрузки и измеряет отдельный аспект поведения системы. Бенчмарки запускаются последовательно в порядке **hackbench**, **sysbench**, **schbench** с периодами остывания между фазами для предотвращения термального дросселирования. Порядок планировщиков рандомизируется между прогонами, что исключает систематическое смещение из-за фоновых процессов и теплового состояния.

hackbench. Утилита из пакета **rt-tests**, создающая группы процессов, обменивающихся сообщениями через каналы (pipes) или сокеты. Нагрузка характеризуется массовым порождением короткоживущих задач и интенсивным межпроцессным взаимодействием, что нагружает механизмы пробуждения, миграции и балансировки планировщика. Измеряемая метрика — суммарное время выполнения (wall-clock time). Меньшее значение соответствует более эффективной работе планировщика при высоком темпе создания и переключения задач.

sysbench. Многопоточный бенчмарк с модулем **oltp_read_only**, выполняющий транзакции чтения к базе данных PostgreSQL. Нагрузка моделирует серверный сценарий со смешанным профилем: потоки чередуют вычисления (разбор запроса) и ввод-вывод (ожидание ответа от базы данных), что создаёт характерный паттерн коротких пробуждений. Измеряемые метрики — число транзакций в секунду (TPS) и число запросов в секунду (QPS). Большее значение соответствует лучшей работе планировщика при нагрузке с частыми блокировками на ввод-вывод.

schbench. Бенчмарк задержек планирования, разработанный в Meta для воспроизведения характеристик веб-нагрузки. Рабочие потоки выполняют искусственные запросы, состоящие из двух фаз сна (имитация сети и диска) и матричных вычислений. Потоки-координаторы ставят задания в очередь и ожидают результатов. Бенчмарк целенаправленно нагружает три аспекта планировщика: способность утилизировать все ядра, поддержание длительных квантов без вытеснения и минимизацию задержки пробуждения. Измеряемые метрики — латентность пробуждения (p50, p99, p99.9), латентность обслуживания запроса (p50, p99, p99.9) и число запросов в секунду (RPS). Данный бенчмарк наиболее чувствителен к качеству решений о размещении задач на

ядрах, поскольку вытеснение во время матричных вычислений приводит к захвату спинлока на чужом ядре и деградации пропускной способности.

Помимо метрик самих бенчмарков, на протяжении всего эксперимента параллельно работает инструмент `sched_latency`, реализованный как eBPF-программа. Он измеряет восемь категорий задержек планирования через точки трассировки ядра: задержку от пробуждения до начала исполнения (`schedule delay`), время нахождения в очереди готовых задач (`runqueue latency`), задержку пробуждения до постановки в очередь (`wakeup latency`), задержку вытеснения (`preemption latency`), задержку пробуждения простаивающего ядра (`idle wakeup`), задержку миграции между ядрами (`migration latency`), длительность непрерывного исполнения задачи (`slice duration`) и длительность добровольного сна (`sleep duration`). Дополнительно из `/proc/stat` и `/proc/schedstat` снимаются утилизация процессора, частота контекстных переключений и число квантов в секунду, а из подсистемы Intel RAPL — потребляемая мощность пакета.

Каждый эксперимент проводится при трёх уровнях нагрузки: `light`, `moderate` и `stress`, различающихся числом групп `hackbench`, числом потоков `sysbench` и параметрами `schbench`. На каждом уровне выполняется шесть прогонов ($N = 6$) с рандомизированным порядком планировщиков. По результатам прогонов вычисляются средние значения, стандартные отклонения и 95%-доверительные интервалы по критерию Стьюдента, что обеспечивает статистически корректное сравнение при малых выборках.

Совокупность бенчмарков покрывает три принципиально различных сценария нагрузки: массовое порождение и переключение задач (`hackbench`), серверная нагрузка с блокировками на ввод-вывод (`sysbench`) и латентно-чувствительная вычислительная нагрузка с длительными квантами (`schbench`). Такой набор позволяет оценить поведение планировщика как в условиях высокого темпа переключений, так и при необходимости минимизировать задержку пробуждения на гетерогенном оборудовании.

5 Анализ результатов эксперимента

5.1 Результаты при лёгкой нагрузке

В режиме лёгкой нагрузки каждый бенчмарк запускается с минимальными параметрами параллелизма, при которых число активных задач не превышает числа доступных ядер. Такой режим позволяет оценить качество решений планировщика в условиях, когда конкуренция за ядра минимальна и определяющим фактором становится выбор типа ядра для каждой задачи.

Результаты бенчмарка `hackbench` представлены в таблице 5.1.

Таблица 5.1: `hackbench`, лёгкая нагрузка (время выполнения, секунды)

Планировщик	Среднее	Ст. откл.	95%-ДИ
A1349 (s4)	1,737	0,013	[1,704; 1,769]
s3 (EEVDF/ext)	1,838	0,034	[1,752; 1,923]
default (EEVDF)	1,900	0,012	[1,870; 1,931]
LAVD	1,981	0,012	[1,953; 2,010]

Планировщик A1349 показывает наименьшее время выполнения `hackbench` среди всех четырёх планировщиков. Преимущество над штатным EEVDF составляет 8,6% (1,737 с против 1,900 с), при этом доверительные интервалы не пересекаются, что свидетельствует о статистической значимости различия. Результат объясняется способностью аукционной модели направлять короткоживущие задачи `hackbench` преимущественно на Р-ядра, где они завершаются быстрее.

Результаты `sysbench` приведены в таблице 5.2.

Таблица 5.2: `sysbench`, лёгкая нагрузка (транзакции в секунду)

Планировщик	Среднее TPS	Ст. откл.	95%-ДИ
default (EEVDF)	4246	18	[4201; 4292]
LAVD	4205	36	[4116; 4294]
s3 (EEVDF/ext)	3793	201	[3295; 4292]
A1349 (s4)	3518	1142	[680; 6356]

На бенчмарке `sysbench` планировщик A1349 уступает штатному EEVDF на 17,1%. Высокое стандартное отклонение ($\sigma = 1142$) указывает на нестабильность работы при данном типе нагрузки. Поскольку `sysbench` генерирует потоки со смешанным профилем (вычисления и ввод-вывод), аукционный механизм при малом числе потоков не получает достаточной информации для оптимального назначения, что проявляется в повышенной дисперсии.

Результаты schbench приведены в таблице 5.3.

Таблица 5.3: schbench, лёгкая нагрузка (латентность пробуждения, мкс)

Планировщик	p50	p99	p99.9	Ср. RPS
s3 (EEVDF/ext)	2	5	59	1668
A1349 (s4)	2,7	155	186	2243
default (EEVDF)	14,7	152	185	591
LAVD	38,3	170	208	678

По пропускной способности (средний RPS) планировщик A1349 превосходит все остальные планировщики в 3,8 раза относительно штатного EEVDF (2243 против 591). Медианная задержка пробуждения (p50) у A1349 составляет 2,7 мкс, что сопоставимо с результатом s3 (2 мкс) и значительно лучше штатного EEVDF (14,7 мкс). Хвостовые латентности p99 и p99.9 у A1349 и EEVDF практически идентичны (155 мкс и 186 мкс против 152 мкс и 185 мкс). Высокая пропускная способность A1349 при сопоставимых хвостовых латентностях объясняется тем, что при низкой нагрузке вычислительные фазы schbench назначаются на Р-ядра, что сокращает длительность каждого запроса без увеличения задержки пробуждения.

5.2 Результаты при стрессовой нагрузке

В режиме стрессовой нагрузки число активных задач многократно превышает число доступных ядер, что создаёт интенсивную конкуренцию за вычислительные ресурсы. Данный режим позволяет оценить работу планировщика в условиях перегрузки, где критически важны справедливость распределения и эффективность балансировки между кластерами ядер.

Результаты hackbench при стрессовой нагрузке представлены в таблице 5.4.

Таблица 5.4: hackbench, стрессовая нагрузка (время выполнения, секунды)

Планировщик	Среднее	Ст. откл.	95%-ДИ
A1349 (s4)	8,547	0,030	[8,474; 8,621]
s3 (EEVDF/ext)	9,176	0,072	[8,998; 9,354]
default (EEVDF)	9,344	0,062	[9,189; 9,498]
LAVD	9,732	0,079	[9,536; 9,928]

Планировщик A1349 сохраняет лидерство на hackbench при стрессовой нагрузке с преимуществом 8,5% над штатным EEVDF (8,547 с против 9,344 с). Доверительные интервалы не пересекаются ни с одним конкурентом. При высокой конкуренции за ядра аукционный механизм эффективно направляет короткие задачи hackbench на Р-ядра, а длительные — на Е-ядра, что приводит к более быстрому освобождению ресурсов и сокращению общего времени выполнения.

Результаты sysbench при стрессовой нагрузке приведены в таблице 5.5.

Таблица 5.5: sysbench, стрессовая нагрузка (транзакции в секунду)

Планировщик	Среднее TPS	Ст. откл.	95%-ДИ
default (EEVDF)	67 373	939	[65 042; 69 705]
LAVD	51 244	1060	[48 610; 53 877]
A1349 (s4)	49 979	318	[49 190; 50 768]
s3 (EEVDF/ext)	41 803	998	[39 324; 44 282]

На sysbench при стрессовой нагрузке штатный EEVDF значительно опережает все sched_ext планировщики. Планировщик A1349 достигает 74,2% пропускной способности EEVDF (49 979 TPS против 67 373 TPS). Отставание sched_ext планировщиков на данном бенчмарке объясняется двумя факторами. Во-первых, дополнительные накладные расходы на BPF-вызовы при высоком темпе переключений контекста. Во-вторых, штатный EEVDF напрямую интегрирован с подсистемой wake_affine, позволяющей пробуждать задачу на ядре, хранящем горячие данные кэша, тогда как sched_ext реализации вынуждены воспроизводить эту оптимизацию самостоятельно. При этом стандартное отклонение A1349 минимально среди всех планировщиков ($\sigma = 318$), что указывает на стабильность работы.

Результаты schbench при стрессовой нагрузке представлены в таблице 5.6.

Таблица 5.6: schbench, стрессовая нагрузка (латентность пробуждения, мкс)

Планировщик	p50	p99	p99.9	Ср. RPS
A1349 (s4)	406	15 915	47 637	8423
LAVD	3735	20 789	59 872	166
default (EEVDF)	4589	11 131	17 163	5645
s3 (EEVDF/ext)	9147	16 112	16 672	8639

При стрессовой нагрузке планировщик A1349 демонстрирует медианную задержку пробуждения 406 мкс, что в 11,3 раза ниже, чем у штатного EEVDF (4589 мкс). Средний RPS составляет 8423, что на 49,2% выше EEVDF (5645) и сопоставимо с s3 (8639). Однако хвостовые латентности p99 и p99.9 у A1349 составляют 15,9 мс и 47,6 мс соответственно, что выше, чем у штатного EEVDF (11,1 мс и 17,2 мс). Данный компромисс объясняется стратегией A1349: аукционный механизм агрессивно назначает задачи на Р-ядра при пробуждении, сокращая медианную задержку, однако задачи с исчерпанным бюджетом вынуждены ожидать Е-ядро, что увеличивает хвост распределения.

Латентность обслуживания запросов schbench представлена в таблице 5.7.

По латентности обслуживания запросов A1349 показывает стабильные результаты: p99 составляет 62,3 мс, а p99.9 — 72,8 мс. Штатный EEVDF при

Таблица 5.7: schbench, стрессовая нагрузка (латентность запроса, мкс)

Планировщик	p50	p99	p99.9
default (EEVDF)	6643	444 245	1 670 485
A1349 (s4)	8699	62 272	72 832
s3 (EEVDF/ext)	8763	22 347	24 907
LAVD	7587	37 464 747	38 207 488

стрессовой нагрузке демонстрирует катастрофическую деградацию хвостовых латентностей: p99 достигает 444 мс, а p99.9 — 1,67 с. LAVD показывает ещё худшие результаты с латентностями порядка десятков секунд. Данное поведение указывает на то, что при перегрузке штатный EEVDF и LAVD допускают голодание отдельных задач в очереди, тогда как аукционный механизм A1349 обеспечивает более равномерное распределение ресурсов благодаря бюджетным ограничениям и VCG-платежам.

5.3 Результаты при умеренной нагрузке

Режим умеренной нагрузки занимает промежуточное положение: число активных задач сопоставимо с числом ядер, но часть из них выполняется одновременно, создавая умеренную конкуренцию за ресурсы.

Результаты hackbench при умеренной нагрузке представлены в таблице 5.8.

Таблица 5.8: hackbench, умеренная нагрузка (время выполнения, секунды)

Планировщик	Среднее	Ст. откл.	95%-ДИ
s3 (EEVDF/ext)	8,716	0,031	[8,640; 8,793]
default (EEVDF)	9,655	0,011	[9,628; 9,683]
LAVD	10,121	0,039	[10,023; 10,218]
A1349 (s4)	12,606	0,220	[12,060; 13,152]

При умеренной нагрузке планировщик A1349 показывает наихудший результат на hackbench (12,6 с), уступая штатному EEVDF на 30,6%. Высокое время выполнения и значительная дисперсия ($\sigma = 0,220$) свидетельствуют о неоптимальном поведении аукционного механизма в данном режиме. При умеренной конкуренции механизм VCG-платежей расходует бюджеты задач быстрее, чем они восстанавливаются, что вынуждает часть задач ожидать E-ядро вместо немедленного назначения на P-ядро. Этот эффект особенно выражен на hackbench, где массовое порождение задач исчерпывает бюджеты групп одновременно.

Результаты sysbench при умеренной нагрузке приведены в таблице 5.9.

На sysbench планировщик A1349 занимает третье место с 13 537 TPS, уступая штатному EEVDF на 18,3%. Широкий доверительный интервал ([6639; 20 435])

Таблица 5.9: sysbench, умеренная нагрузка (транзакции в секунду)

Планировщик	Среднее TPS	Ст. откл.	95%-ДИ
default (EEVDF)	16 566	32	[16 487; 16 645]
LAVD	14 233	17	[14 191; 14 276]
A1349 (s4)	13 537	2777	[6639; 20 435]
s3 (EEVDF/ext)	12 517	96	[12 279; 12 756]

обусловлен высокой дисперсией между прогонами. Тем не менее медианный результат A1349 превышает результат s3 на 8,1%, что подтверждает положительное влияние учёта гетерогенности при обработке смешанных нагрузок.

Результаты schbench при умеренной нагрузке представлены в таблице 5.10.

Таблица 5.10: schbench, умеренная нагрузка (латентность пробуждения, мкс)

Планировщик	p50	p99	p99.9	Ср. RPS
LAVD	5	881	1437	4281
A1349 (s4)	6	1562	8411	7535
s3 (EEVDF/ext)	23	2375	2815	8585
default (EEVDF)	85	3300	4733	5700

На schbench при умеренной нагрузке планировщик A1349 показывает медианную задержку пробуждения 6 мкс, что в 14 раз ниже штатного EEVDF (85 мкс). По средней пропускной способности A1349 достигает 7535 RPS, превышая EEVDF на 32,2% и уступая лишь s3 (8585). Хвостовая латентность p99.9 у A1349 составляет 8,4 мс, что выше, чем у LAVD (1,4 мс) и s3 (2,8 мс), но значительно лучше штатного EEVDF (4,7 мс) в перцентиле p99.

5.4 Обсуждение результатов

Проведённый эксперимент позволяет сформулировать основные закономерности поведения планировщика A1349 на гетерогенной платформе в сравнении со штатным EEVDF, его sched_ext реимплементацией (s3) и планировщиком LAVD.

Преимущества модели A1349. По бенчмарку haskbench планировщик A1349 лидирует при лёгкой и стрессовой нагрузке, обеспечивая сокращение времени выполнения на 8,5–8,6% относительно штатного EEVDF. Данный бенчмарк создаёт множество короткоживущих задач с интенсивным межпроцессным взаимодействием, что соответствует ситуации, в которой аукционный механизм наиболее эффективен: каждая задача имеет малую длину контракта (l_i), и назначение на Р-ядро даёт непропорционально большой выигрыш по времени завершения.

По пропускной способности schbench (RPS) планировщик A1349 стабильно превосходит штатный EEVDF на всех уровнях нагрузки: в 3,8 раза при лёгкой, на 32,2% при умеренной и на 49,2% при стрессовой нагрузке. Медианная задержка пробуждения p_{50} у A1349 на порядок ниже, чем у EEVDF: 2,7 мкс против 14,7 мкс (лёгкая), 6 мкс против 85 мкс (умеренная) и 406 мкс против 4589 мкс (стрессовая). Это объясняется свойством модели: задачи с высокой ценностью v_i (чувствительные к задержке) получают Р-ядра по результатам аукциона, что сокращает как задержку пробуждения, так и длительность обслуживания запроса.

Ограничения модели A1349. На бенчмарке sysbench планировщик A1349 стабильно уступает штатному EEVDF на 17–26% в зависимости от уровня нагрузки. Аналогичную деградацию демонстрируют все sched_ext планировщики (s3 уступает на 10–38%), что указывает на системные накладные расходы фреймворка при нагрузках с высоким темпом переключений контекста и короткими периодами сна. Дополнительным фактором является отсутствие в sched_ext встроенной интеграции с wake_affine, оптимизирующей локальность кэша при пробуждении.

При умеренной нагрузке hackbench планировщик A1349 показывает наихудший результат, уступая EEVDF на 30,6%. Данный режим выявляет уязвимость бюджетного механизма: при массовом одновременном порождении задач в условиях умеренной конкуренции бюджеты исчерпываются быстрее, чем поступают новые задачи, способные утилизировать освободившиеся Р-ядра. Проблема может быть решена динамической калибровкой скорости расхода бюджета в зависимости от текущей загрузки системы.

Хвостовые латентности p_{99} и $p_{99.9}$ schbench при стрессовой нагрузке у A1349 выше, чем у штатного EEVDF (15,9 мс и 47,6 мс против 11,1 мс и 17,2 мс). Данный компромисс является следствием бюджетных ограничений: задачи, израсходовавшие бюджет на Р-ядрах, перенаправляются на Е-ядра, где обслуживание длится дольше. Аналогичный эффект предсказывает теория (глава 2): при $\gamma < \sigma$ Е-ядро обходится дешевле, но контракт длиннее в σ раз.

Влияние свойств модели на результаты. Учёт гетерогенности (параметр σ модели) проявляется в сокращении времени hackbench и медианной задержки schbench: зная соотношение производительностей ядер, механизм назначает латентно-чувствительные задачи на Р-ядра и фоновые — на Е-ядра, что невозможно в рамках штатного EEVDF, оперирующего единым виртуальным временем для всех ядер. VCG-распределение обеспечивает приоритет задачам с наибольшей эффективной стоимостью $\phi_P(\theta_i)$ без явного задания приоритетов, а бюджетные ограничения B_i предотвращают монополизацию Р-ядер отдельными задачами, что подтверждается стабильностью хвостовых латентностей обслуживания запросов при стрессовой нагрузке (72,8 мс у A1349 против 1,67 с

у EEVDF в p99.9).

Ограничения эксперимента. Малый объём выборки ($n = 3$ для каждого планировщика на каждом уровне нагрузки) ограничивает статистическую мощность сравнения, что проявляется в широких доверительных интервалах для sysbench. Эксперимент проведён на единственной аппаратной платформе (Intel Arrow Lake-S), и переносимость результатов на архитектуры ARM big.LITTLE и RISC-V требует дополнительной проверки. Кроме того, использованные бенчмарки не покрывают нагрузки реального времени и GPU-ориентированные сценарии, для которых поведение модели может существенно отличаться.

Заключение

Список литературы

- [1] John Lions A Commentary on the Sixth Edition UNIX Operating System The University of New South Wales 1977
- [2] Mike Kravetz, Hubertus Franke, Shailabh Nagar, Rajan Ravindran Enhancing Linux Scheduler Scalability 2001
- [3] Robert Love Linux Kernel Development 2nd ed Novell Press, 2005
- [4] C. S. Wong, I. K. T. Tan, R. D. Kumari, J. W. Lam, W. Fun Fairness and Interactive Performance of O(1) and CFS Linux Kernel Schedulers International Symposium on Information Technology 2008
- [5] Ion Stoica, Hussein Abdel-Wahab Earliest Eligible Virtual Deadline First : A Flexible and Accurate Mechanism for Proportional Share Resource Allocation Old Dominion University 1996
- [6] Greenhalgh P. Big.LITTLE Processing with ARM Cortex-A15 & Cortex-A7 ARM White Paper, 2011
- [7] Rotem E. et al. Alder Lake Architecture 2021 IEEE Hot Chips 33 Symposium (HCS), Palo Alto, CA, USA, 2021, pp. 1–23, doi: 10.1109/HCS52781.2021.9567097
- [8] Conti F., Paulin G., Garofalo A., Rossi D., Pullini A., Benini L. Marsellus: A Heterogeneous RISC-V AI-IoT End-Node SoC with 2-to-8b DNN Acceleration and 30%-Boost Adaptive Body Biasing IEEE Journal of Solid-State Circuits, 2024
- [9] Energy Aware Scheduling [Электронный ресурс]. Linux Kernel Documentation. Режим доступа: <https://www.kernel.org/doc/html/latest/scheduler/sched-energy.html> – Дата обращения 14.11.2025
- [10] Corbet J. The BPF extensible scheduler class LWN.net, 30 November 2022 Режим доступа: <https://lwn.net/Articles/916291/> – Дата обращения 20.11.2025
- [11] Humphries J. T., Natu N., Chaugule A., Weisse O., Rhoden B., Don J., Rizzo L., Rombakh O., Turner P., Kozyrakis C ghOST: Fast & Flexible User-Space Delegation of Linux Scheduling SOSP 2021
- [12] Tang Q., Gao X., Li G., Lin J Optimizing Linux Scheduling Based on Global Runqueue with SCX IEEE International Conference SMC 2024
- [13] Xu J., Wolff D., Yi H. X., Li J., Roychoudhury A. Concurrency Testing in the Linux Kernel via eBPF arXiv 2025

- [14] Mao J., Ujcich B. E., Ma S Rethinking Provenance Completeness with a Learning-Based Linux Scheduler arXiv 2025
- [15] Wang X., Jia S., Huang Z., Cao J., Song M Mixture-of-Schedulers: An Adaptive Scheduling Agent as a Learned Router for Expert Policies arXiv 2025
- [16] Galin M., Hedblom A Linux Kernel Scheduler Evaluation for Performance-Critical Telecom Workloads Linköping University 2025
- [17] Van Craeynest K., Jaleel A., Eeckhout L., Narvaez P., Emer J. S Scheduling Heterogeneous Multi-Cores through Performance Impact Estimation (PIE) ISCA, ACM, 2012, pp. 213–224
- [18] Sano R. A Dynamic Mechanism Design for Scheduling with Different Use Lengths Institute of Economic Research, Kyoto University, 2013
- [19] Leon V., Etesami S. R. Online Learning for Dynamic Vickrey-Clarke-Groves Mechanism in Unknown Environments arXiv:2506.19038, 2025
- [20] Joseph Y-T. Leung Handbook of Scheduling: Algorithms, Models, and Performance Analysis CRC Press 2004
- [21] Michelle Galin, Anna Hedblom Linux Kernel Scheduler Evaluation for Performance-Critical Telecom Workloads Linköping University 2025
- [22] Jinsong Mao, Benjamin E. Ujcich, Shiqing Ma Rethinking Provenance Completeness with a Learning-Based Linux Scheduler arXiv 2025
- [23] Xinbo Wang, Shian Jia, Ziyang Huang, Jing Cao, Mingli Song Mixture-of-Schedulers: An Adaptive Scheduling Agent as a Learned Router for Expert Policies arXiv 2025
- [24] A1349 Pavel Shago [Электронный ресурс]. – Режим доступа: <https://github.com/shgpavel/A1349> – Дата обращения 16.12.2025.
- [25] Andrew S. Tanenbaum, Herbert Bos MODERN OPERATING SYSTEMS Pearson Education 2023
- [26] Liz Rice Learning eBPF O'Reilly Media, Inc 2023
- [27] Torvalds L. Linux Kernel [Электронный ресурс] / L. Torvalds. – Режим доступа: <https://github.com/torvalds/linux> – Дата обращения 04.12.2025.
- [28] sched-ext/scx [Электронный ресурс]. – Режим доступа: <https://github.com/sched-ext/scx> – Дата обращения 07.12.2025.
- [29] lkml [Электронный ресурс]. – Режим доступа: <https://lkml.org> – Дата обращения 24.11.2025.